

React 19 · Flask · MongoDB · Tailwind CSS v4

A complete professional reference guide covering architecture, authentication, database design, UI components, API endpoints, security model and design system decisions.

Table of Contents

1. Project Overview
2. Technology Stack & Dependencies
3. Project Folder Structure
4. Frontend Architecture (React)
5. Backend Architecture (Flask)
6. Database Design (MongoDB)
7. Authentication & Security (JWT)
8. API Endpoint Reference
9. Design System & Tailwind Theme
10. Data Flow — End to End
11. Planned Features

1. Project Overview

Task Manager is a professional full-stack web application that allows authenticated users to create, manage, and track their personal tasks. It features a dark-themed, gradient-driven dashboard with a modular component structure built entirely in React. The backend is a RESTful API built with Python Flask that communicates with a MongoDB NoSQL database. All user data is isolated by account using JWT-based authentication.

Key Characteristics:

- Single Page Application (SPA) — no full page reloads.
- Stateless backend — the server does not store sessions, all identity is in the JWT token.
- User-isolated data — each user can only access their own tasks in MongoDB.
- Persistent login — token stored in localStorage survives browser refresh.
- Dark professional UI with custom Tailwind v4 theme variables.

2. Technology Stack & Dependencies

Frontend (react/frontend/package.json)

Package	Version	Purpose
react	^19.2.5	Core UI library
react-dom	^19.2.5	DOM rendering for React
vite	^6.x	Build tool and dev server
tailwindcss	^4.2.4	Utility-first CSS framework
@tailwindcss/vite	^4.2.4	Tailwind v4 Vite plugin integration

Backend (backend/requirements.txt)

Package	Purpose
flask	Web framework — handles HTTP routes and responses
flask-cors	Allows React (port 5173) to talk to Flask (port 5000)
pymongo	Official MongoDB driver for Python
python-dotenv	Loads secrets from a .env file safely
bcrypt	Password hashing — never stores plain text passwords
PyJWT	Creates and verifies JSON Web Tokens
reportlab	PDF generation (used for this report)

3. Project Folder Structure

```
project-root/
├── backend/
│   ├── app/
│   │   ├── __init__.py    ← Flask app factory, registers Blueprints & CORS
│   │   ├── models.py      ← MongoDB client, db, tasks + users collections
│   │   └── routes.py       ← All API routes: Auth (/login, /register) + Tasks
│   ├── requirements.txt   ← Python package list
│   └── run.py             ← Entry point: starts Flask on port 5000
├── react/frontend/
│   ├── src/
│   │   ├── App.jsx        ← Main layout: TopBar + Sidebar + Dashboard
│   │   ├── index.css      ← Tailwind import + custom @theme variables
│   │   └── components/
│   │       ├── TopBar.jsx  ← Header: logo, auth modal, user profile
│   │       └── LeftSideBar.jsx ← Navigation: Dashboard, Notes, Gallery...
│   ├── index.html         ← HTML entry point (mounts React root)
│   └── package.json        ← Node dependencies and scripts
├── generate_pdf.py         ← Script that generates this PDF report
├── DOCUMENTATION.md       ← Human-readable project docs
├── AUTH_REPORT.md         ← Detailed authentication explanation
└── README.md              ← Project overview with dashboard preview
```

4. Frontend Architecture (React)

App.jsx — The Layout Orchestrator

App.jsx is the root component that controls the entire page layout. It uses a Flexbox column structure: TopBar on top, then a row containing LeftSidebar and the main content area side by side. The key CSS principle is h-screen + overflow-hidden on the wrapper, and overflow-y-auto only on the main content — this creates a true app-like experience where the sidebar and header never scroll.

Key React concepts used:

- `useState` — tracks tasks, `newTaskTitle`, `newTaskDate`, `isAdding`, and user state.
- `useEffect` — runs `fetchTasks()` once on page load to populate the dashboard.
- `fetch()` — used for all HTTP calls (GET, POST, DELETE) to the Flask backend.
- Conditional rendering — shows Auth component if no user is logged in.
- Array methods — `.filter()` removes a task client-side after successful DELETE.

TopBar.jsx — Identity & Auth Modal

TopBar manages the entire authentication UI. It maintains 5 state variables: `isLoggedIn`, `userData`, `showModal`, `isRegister`, and `formData`. On mount, `useEffect` checks `localStorage` for a saved token and user — if found, the user is automatically logged back in. The modal toggles between Login and Register modes with a single boolean state (`isRegister`).

LeftSidebar.jsx — Navigation

A static navigation component that renders 5 menu items using `.map()`. The active item (Dashboard) is highlighted using a conditional Tailwind class that applies a dusk gradient background. Other items show a warm gradient vertical bar on hover using CSS opacity transitions.

5. Backend Architecture (Flask)

__init__.py — App Factory

Uses the Flask Application Factory pattern. `create_app()` initializes the Flask app, applies CORS (which allows requests from the React dev server on port 5173), and registers both Blueprints: `tasks_bp` and `auth_bp`.

```
from .routes import tasks_bp, auth_bp app.register_blueprint(tasks_bp)
app.register_blueprint(auth_bp)
```

models.py — Database Connection

Connects to MongoDB using the URI from a `.env` file. Defines two collection variables — `tasks_collection` and `users_collection` — that are imported directly into `routes.py` and used for all database operations.

routes.py — All Business Logic

The core of the backend. Divided into two sections: Auth Routes (public) and Task Routes (protected). The file imports `bcrypt`, `jwt`, `datetime`, `os` and `ObjectId` from `bson` for all operations.

token_required Decorator:

```
def token_required(f): def decorated(*args, **kwargs): token =
request.headers.get('Authorization') if not token: return error 401 data =
jwt.decode(token, SECRET_KEY) current_user = users_collection.find_one({'_id':
ObjectId(data['user_id'])}) return f(current_user, *args, **kwargs)
```

This decorator is applied with `@token_required` on every task route. It intercepts the request, reads the Authorization header, decodes the JWT to extract the `user_id`, fetches the user from MongoDB, and passes them as the first argument (`current_user`) into the protected function.

6. Database Design (MongoDB)

MongoDB is a NoSQL document database. Data is stored as JSON-like documents inside collections (equivalent to SQL tables). The database name is 'taskmanager' and contains two collections.

users Collection

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier by MongoDB
email	String	User email address (must be unique)
password	Binary	Bcrypt-hashed password (never plain text)
name	String	Display name shown in the TopBar
created_at	DateTime	UTC timestamp of registration

tasks Collection

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier by MongoDB
title	String	The task description text
done	Boolean	Completion status — False by default
date	String	Optional due date set by the user
user_id	String	Links the task to its owner (from users._id)

The `user_id` field is the key to data isolation. When fetching tasks, the query is: `tasks_collection.find({'user_id': str(current_user['_id'])})` — this ensures each user only ever receives their own data.

7. Authentication & Security (JWT)

Step 1 — Registration

React sends { name, email, password } as JSON to POST /register.

Flask checks if the email already exists in users_collection.

If new, bcrypt.hashpw() converts the plain password into a secure hash.

The hash (not the original password) is stored in MongoDB.

A 201 Created response is returned — user is told to log in.

Step 2 — Login

React sends { email, password } to POST /login.

Flask finds the user by email in users_collection.

bcrypt.checkpw() compares the input password against the stored hash.

If they match, jwt.encode() creates a token containing { user_id, exp }.

The token + user info is returned to React.

React stores token in localStorage.setItem('token', ...) for persistence.

Step 3 — Authenticated Requests

React reads the token from localStorage.

Sends it in every request header: Authorization: Bearer .

@token_required intercepts the request and decodes the token.

If valid, current_user is extracted and passed to the route function.

If expired or invalid, a 401 Unauthorized response is returned.

Security Notes:

- Passwords are NEVER stored or compared as plain text — bcrypt is used.
- JWT tokens expire after 24 hours (timedelta(hours=24)).
- The SECRET_KEY is loaded from a .env file — never hardcoded in production.
- Each task query filters by user_id — cross-user data access is impossible.
- CORS is configured to accept requests from the frontend origin only.

8. API Endpoint Reference

Public Endpoints (No Token Required)

Method	Endpoint	Body (JSON)	Response
POST	/register	{ name, email, password }	201 — { message, id }
POST	/login	{ email, password }	200 — { token, user }

Protected Endpoints (JWT Token Required in Header)

Method	Endpoint	Body (JSON)	Response
GET	/tasks	None	200 — [list of tasks]
POST	/tasks	{ title, date }	201 — { new task object }
PUT	/tasks/<id>	{ done, title, ... }	200 — { updated task }
DELETE	/tasks/<id>	None	200 — { message: Deleted }

Header format for protected requests: Authorization: Bearer eyJhbGciOiJIUzI1...

9. Design System & Tailwind Theme

The application uses Tailwind CSS v4, which allows defining custom design tokens using the `@theme` block in `index.css`. These variables become available as Tailwind utility classes throughout the entire codebase.

Variable	Hex Value	Usage
--color-dusk-start	#6366f1 (Indigo)	Gradient start — notes card, topbar logo
--color-dusk-end	#ec4899 (Pink)	Gradient end — active nav item, logout text
--color-warm-start	#f59e0b (Amber)	Gradient start — documents card, avatar ring
--color-warm-end	#ef4444 (Red)	Gradient end — hover accents
--color-dashboard-bg	#0f172a (Slate 900)	Main page background, sidebar
--color-card-bg	#1e293b (Slate 800)	Card backgrounds, topbar background

Usage in JSX:

```
bg-gradient-to-r from-dusk-start to-dusk-end text-dusk-end bg-dashboard-bg bg-card-bg
```

10. Data Flow — End to End

Step	Actor	Action
1	Browser	User opens http://localhost:5173
2	React	App.jsx checks localStorage for token
3	React	If no token → shows Auth modal (TopBar)
4	User	Enters email + password, submits form
5	React	fetch() POST to http://localhost:5000/login
6	Flask	Checks email in MongoDB users collection
7	Flask	bcrypt.checkpw() verifies the password hash
8	Flask	jwt.encode() creates a 24h token, returns it
9	React	Saves token + user to localStorage
10	React	Updates state: isLoggedIn=true, shows user name
11	React	useEffect calls fetchTasks() with Bearer token
12	Flask	@token_required validates token, finds user
13	MongoDB	Returns tasks where user_id matches current user
14	React	setTasks(data) — dashboard renders task list

11. Planned Features

Feature	Description	Status
Notes Module	Create, read, update, delete personal notes	Planned
Gallery Module	Upload and display images per user account	Planned
Calculator Widget	In-dashboard calculator component	Planned
Documents Section	Upload and manage PDF/DOC files	Planned
Task Priority	Mark tasks as High / Medium / Low	Planned
Due Date Alerts	Notify when a task deadline is approaching	Planned
Task Categories	Group tasks by Work, Personal, School etc.	Planned
Profile Settings	Update name, email, avatar, password	Planned